

Heuristic Query Optimization

Database management Systems

Where is the data Stored

```
SQL> desc emp1;
```

Name	Type
-----	-----
ID	VARCHAR2 (3)
NAME	VARCHAR2 (3)
DEPNO	VARCHAR2 (3)
DESIG	VARCHAR2 (3)

```
INSERT INTO EMPO VALUES ('123', 'SAM', '111', 'CKM');
```

How many Bytes of Data is getting
Inserted here?

How is the Data Stored Internally?

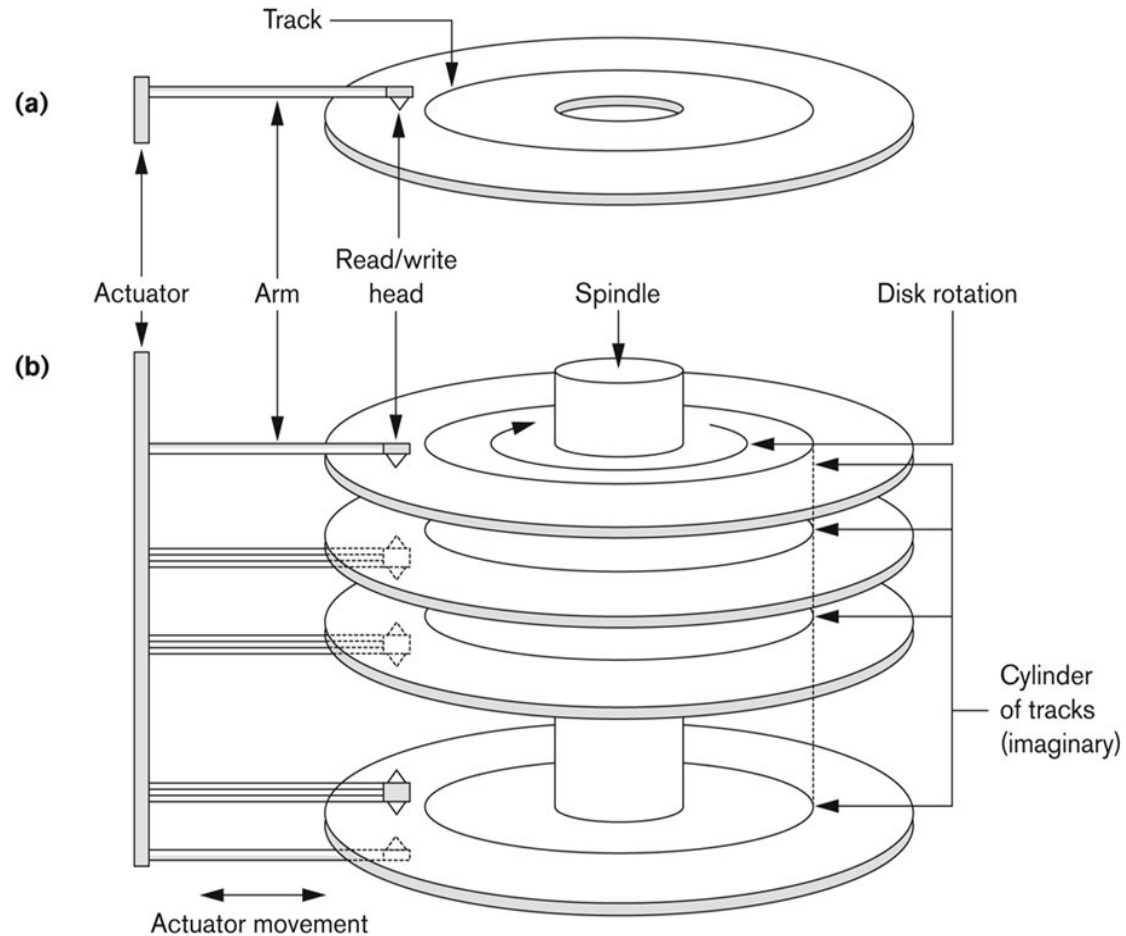
- Data is stored on a Hard



Hard Disk Architecture

Figure 13.1

(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.



Hard Disk Architecture

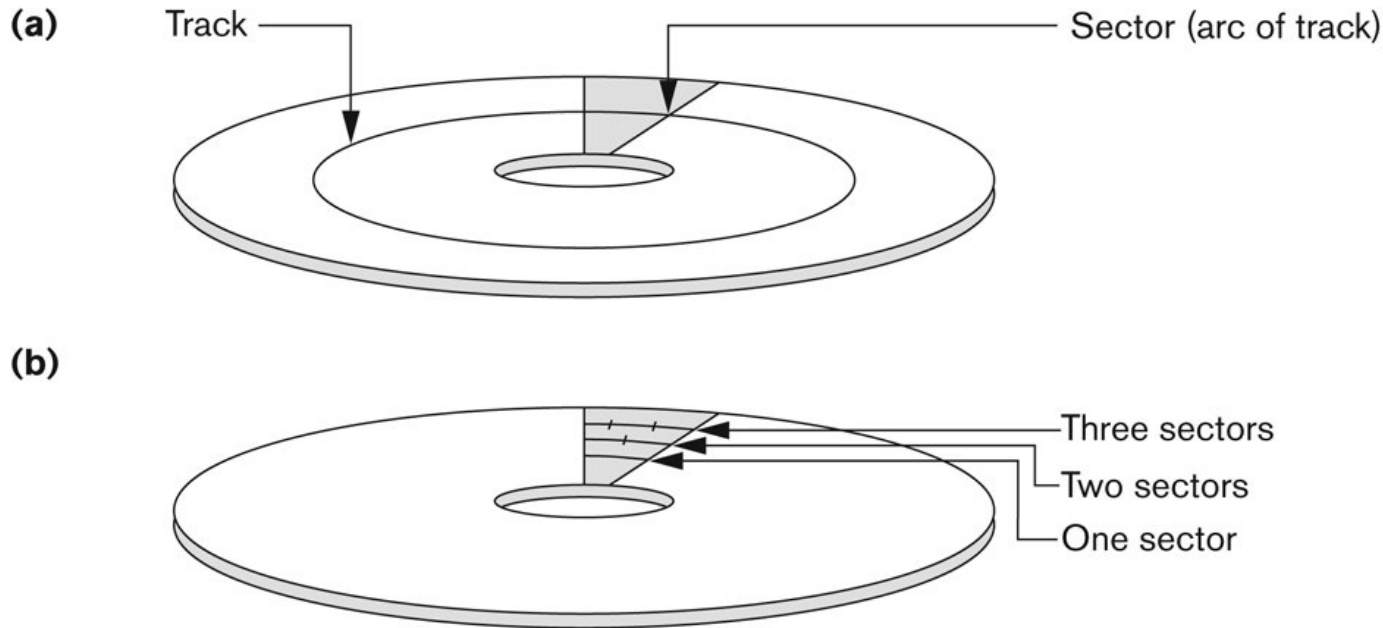


Figure 13.2

Different sector organizations on disk.
(a) Sectors subtending a fixed angle.
(b) Sectors maintaining a uniform recording density.

Disks are divided into concentric circular tracks on each disk surface. Track capacities vary typically from 4 to 50 Kbytes or more

Hard Disk Architecture

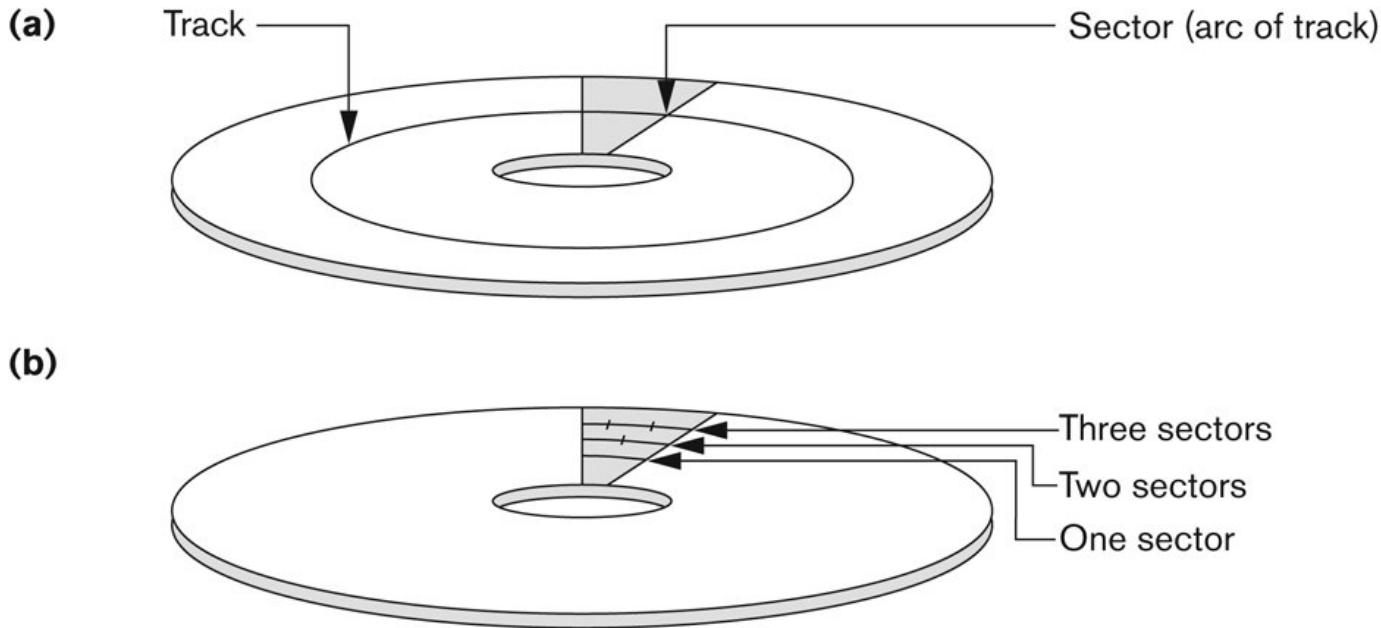


Figure 13.2

Different sector organizations on disk.
(a) Sectors subtending a fixed angle.
(b) Sectors maintaining a uniform recording density.

A track is divided into **blocks**.

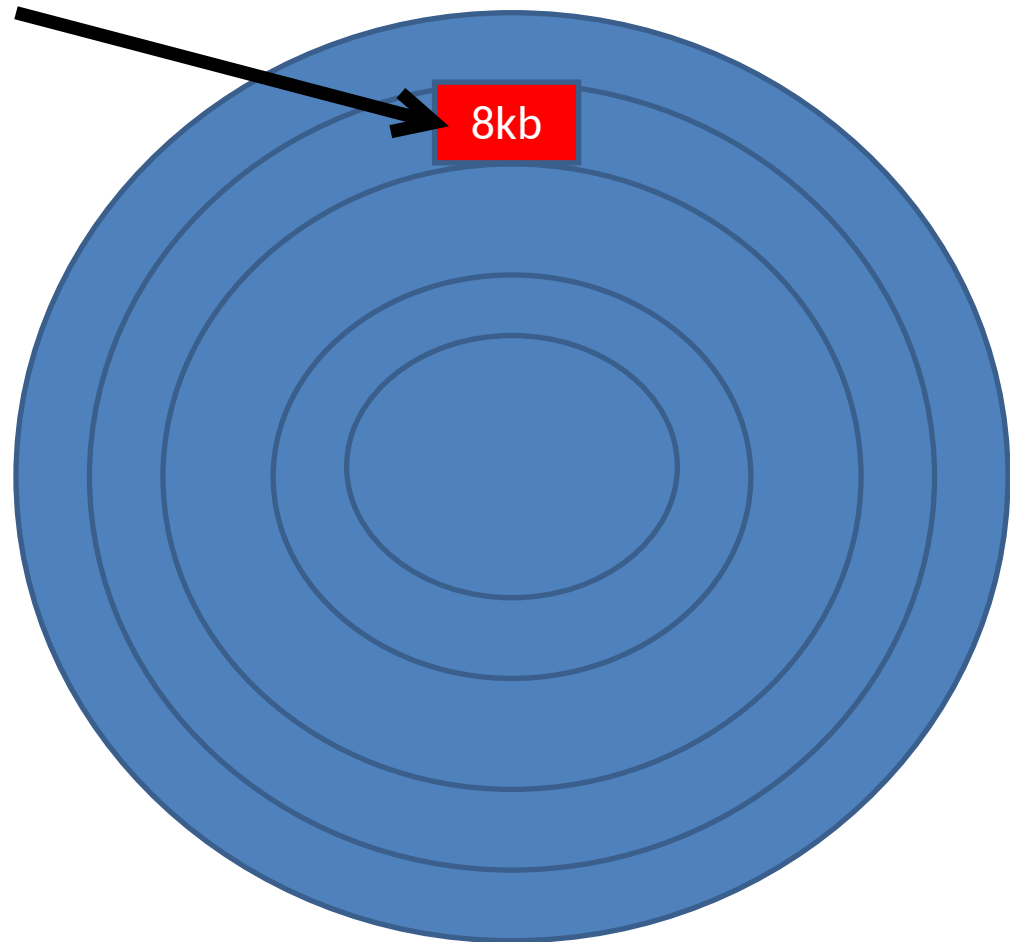
The block size B is fixed for each system.

Typical block sizes range from $B=512$ bytes, $B=4096$ bytes, $B=8192$ Bytes

Whole blocks are transferred between disk and main memory for processing.

How data gets stored

```
INSERT INTO EMPLOYEE  
VALUES ('123', 'SAM', '111');
```



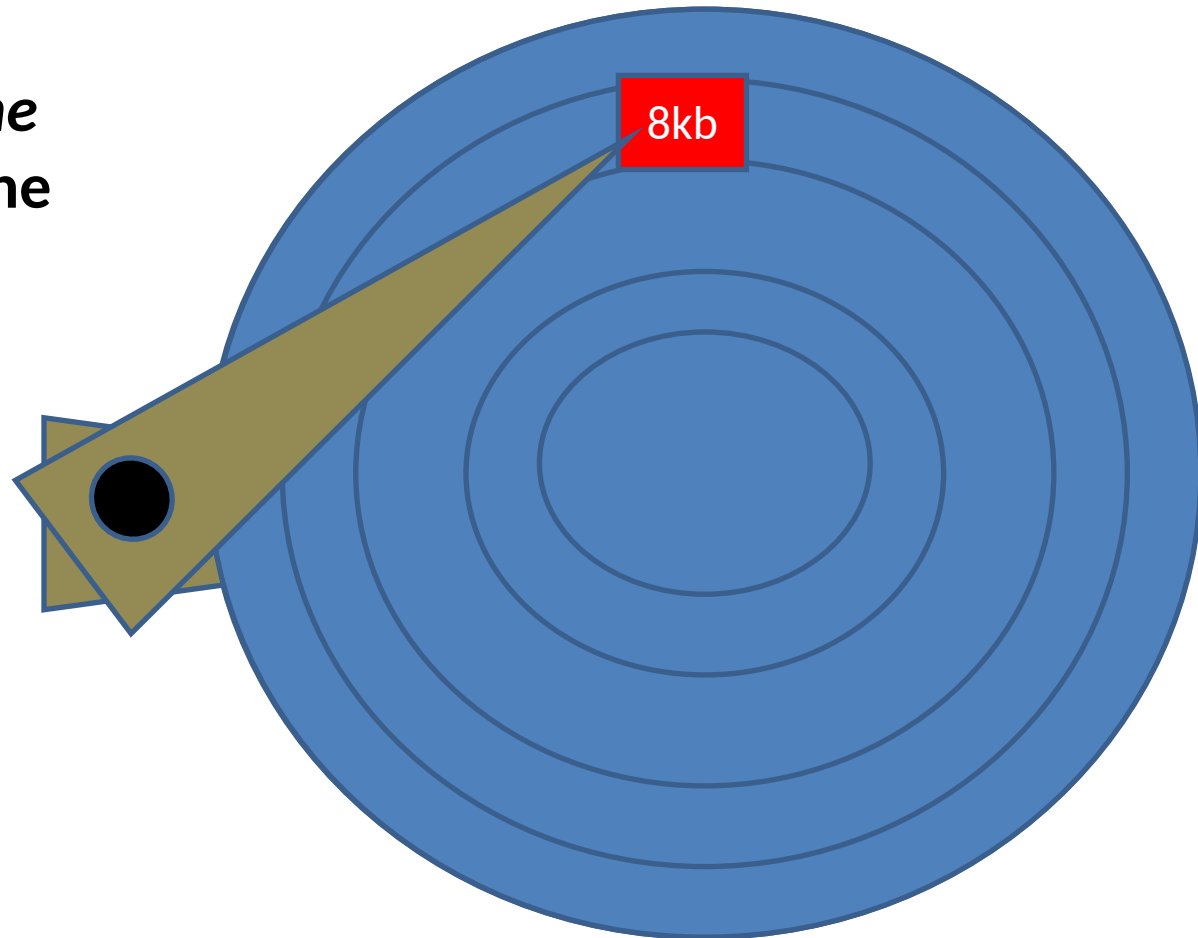
- A block can hold many records
- A block is transferred between disk and main memory.

Seek Time and Latency Time

- Select * from empo;

Seek time is the *time* required to move the disk arm to the required track.

Latency – Desired block rotates into Position under The read/write head



Total Time Needed for an arbitrary block transfer

- Block Transfer Time
Additional time required to transfer the data.
- Total Time Needed =
Seek Time + Latency + Block Transfer Time

We spend some time to read every block of data from the disk

Lets Take an Example

- Consider the two tables with the data given below
- Empo

```
SQL> SELECT * FROM EMP1;
```

ID	NAM	DEP	DES
---	---	---	---
123	SAM	111	AKJ
124	TOM	222	CMU
125	RAM	333	KCM
126	SAM	222	CMU
127	SAM	333	KCM

**Total Bytes
Allocated =
60 Bytes**

Dep Table

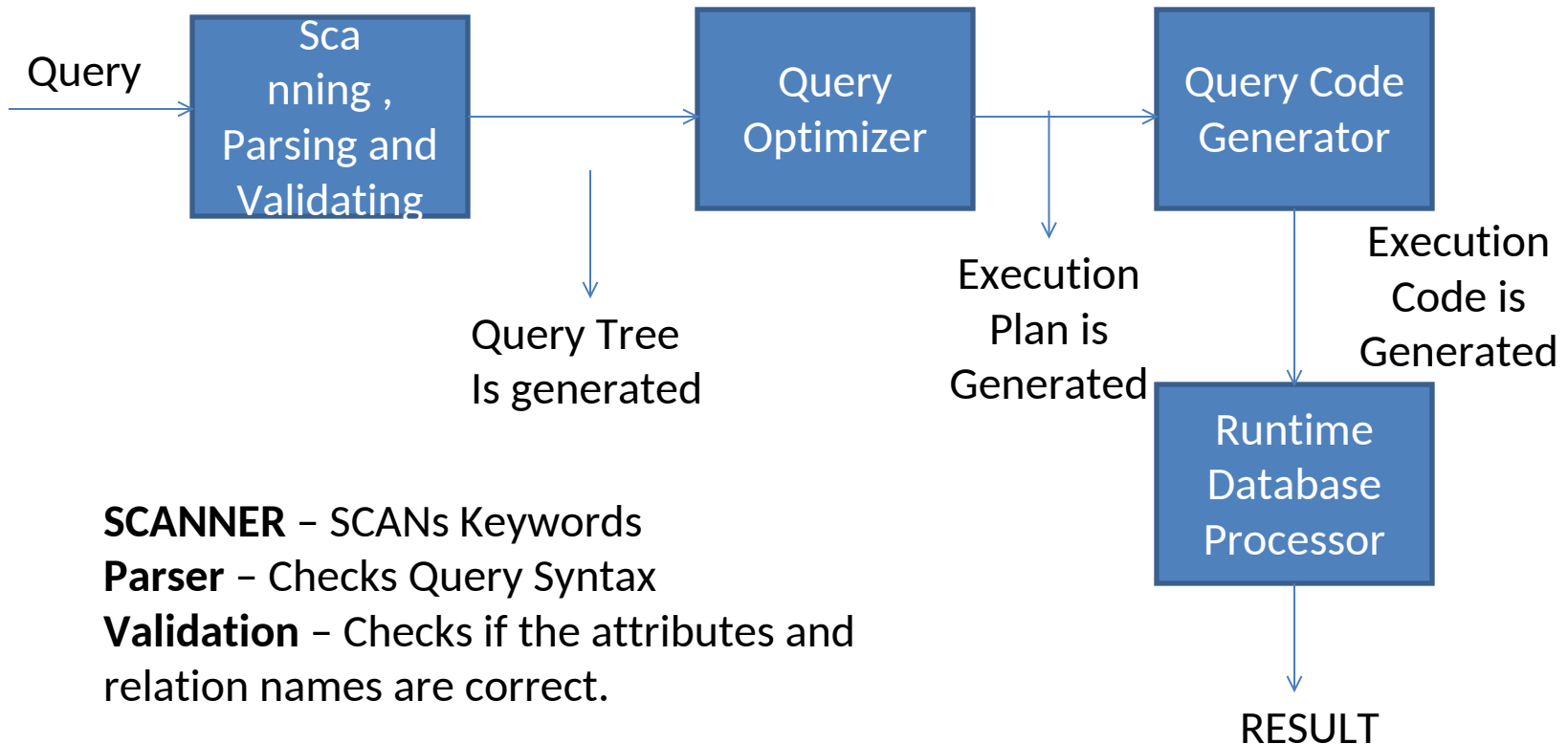
```
SQL> select * from dep1;
```

DNO	DNAM	DLO
---	-----	---
111	SBST	SJT
222	SITE	MBD
333	SMEC	SMV
444	SBST	MBD

**Total Bytes
Allocated =
40 Bytes**

```
Select name,dname from employee,department  
where dep_no=dno and name='SAM' and  
dname='SBST' ;
```

Query Optimization



Initial query Tree

- Parser generates a initial query tree.
- Optimizer optimizes the Query tree using heuristic
- A heuristic is a rule that will work well in most cases but is not guaranteed to work well in every case.

Initial Query Tree



Name,Dname

?

(Dep_no=dno ^ dname = 'SBST' ^ name = 'SAM')



20 Rows
Each Row
22 bytes
Total - 440 Bytes

5 Rows
Each Row
12 bytes

emp1

Total
60 Bytes

4 Rows
Each Row
10 bytes

dep1

Total
40 Bytes

Access From Disk
- 100 Bytes

Viewing Query Plan for Cartesian Product

PLAN_TABLE_OUTPUT

Plan hash value: 2561979186

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		20	440	9 (0)	00:00:01
1	MERGE JOIN CARTESIAN		20	440	9 (0)	00:00:01
2	TABLE ACCESS FULL	DEP1	4	40	3 (0)	00:00:01
3	BUFFER SORT		5	60	6 (0)	00:00:01
4	TABLE ACCESS FULL	EMP1	5	60	2 (0)	00:00:01

PLAN_TABLE_OUTPUT

emp1 cross Join dep1

```
SQL> select * from emp1 cross join dep1;
```

TEMP1

ID	NAM	DEP	DES	DNO	DNAM	DLO
123	SAM	111	AKJ	111	SBST	sjt
124	TOM	222	CMU	111	SBST	sjt
125	RAM	333	KCM	111	SBST	sjt
126	SAM	222	CMU	111	SBST	sjt
127	SAM	333	KCM	111	SBST	sjt
123	SAM	111	AKJ	222	SITE	MB
124	TOM	222	CMU	222	SITE	MB
125	RAM	333	KCM	222	SITE	MB
126	SAM	222	CMU	222	SITE	MB
127	SAM	333	KCM	222	SITE	MB
123	SAM	111	AKJ	333	SMEC	SMV

ID	NAM	DEP	DES	DNO	DNAM	DLO
124	TOM	222	CMU	333	SMEC	SMV
125	RAM	333	KCM	333	SMEC	SMV
126	SAM	222	CMU	333	SMEC	SMV
127	SAM	333	KCM	333	SMEC	SMV
123	SAM	111	AKJ	444	SBST	MB
124	TOM	222	CMU	444	SBST	MB
125	RAM	333	KCM	444	SBST	MB
126	SAM	222	CMU	444	SBST	MB
127	SAM	333	KCM	444	SBST	MB

TEMP 1 Size -
20 Rows with each 22
bytes long.
Total Bytes - 440

? (Dep_no=dno ^ dname = 'SBST' ^ name
= 'SAM')(Temp1)



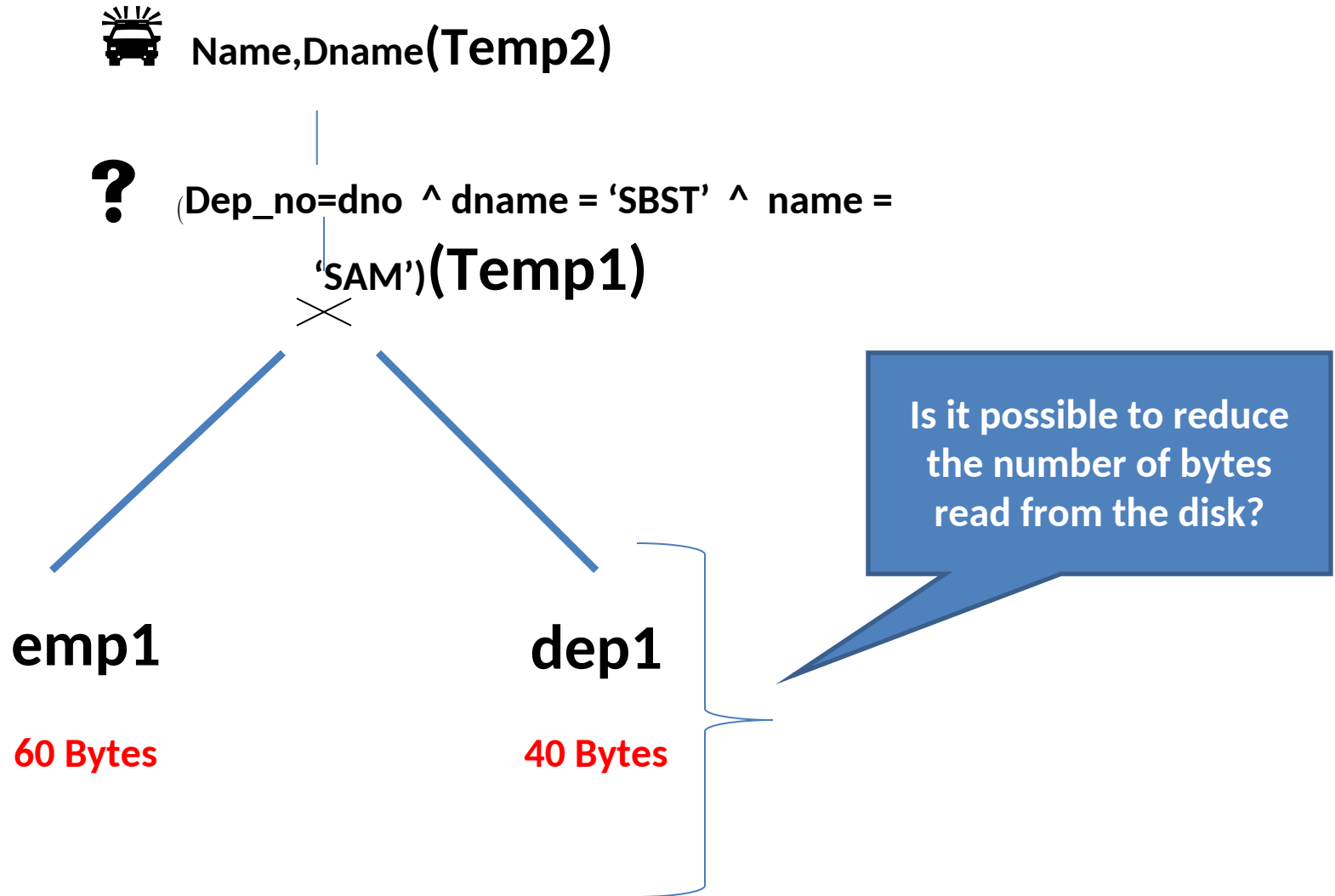
Name,Dname(Temp2)

ID	NAM	DEP	DES	DNO	DNAM	DLO
---	---	---	---	---	-----	---
123	SAM	111	AKJ	111	SBST	SJT

Final Result -

NAM	DNAM
---	-----
SAM	SBST

Initial Query Tree



RULE 1

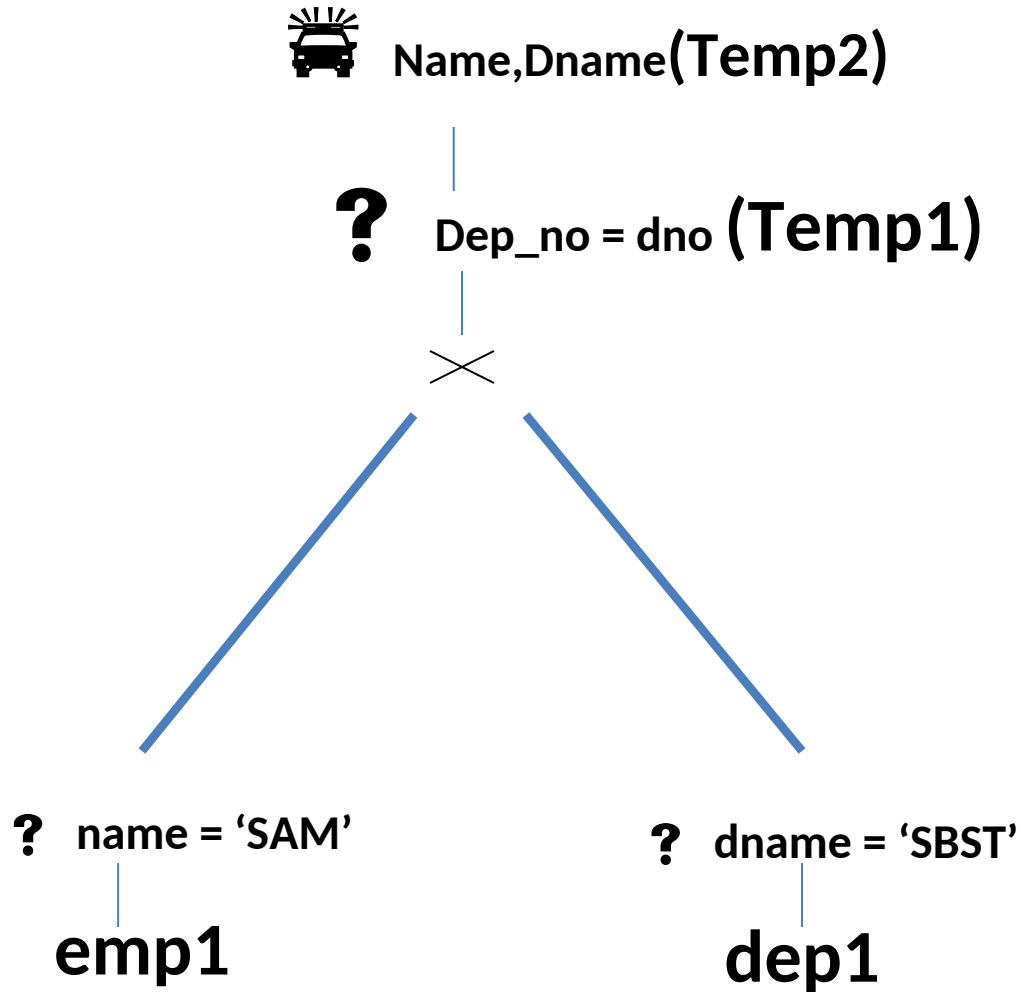
- Cascade of ?
- A conjunctive selection condition can be broken up in to a cascade of individual ? operations

? (Dep_no=dno ^ dname = 'SBST' ^ name = 'SAM')(Temp1)



? Dep_no=dno (? dname = 'SBST' (? name = 'SAM'(Temp1)))

Rule 2 : Move the Select ? as far as down the tree



Only Employees with Name 'SAM'

```
SQL> SELECT * FROM EMP1;
```

ID	NAM	DEP	DES
----	----	----	----
123	SAM	111	AKJ
124	TOM	222	CMU
125	RAM	333	KCM
126	SAM	222	CMU
127	SAM	333	KCM

READ ONLY 36
Bytes

Only Departments with Name 'SBST'

```
SQL> select * from dep1;
```

DNO	DNAM	DLO
---	-----	---
111	SBST	SJT
222	SITE	MBD
333	SMEC	SMV
444	SBST	MBD

READ ONLY 20
Bytes

Rule 2 : Move the Select ? as far as down the tree



Name,Dname(Temp2)

?

Dep_no = dno (Temp1)

6 Rows
Each Row
22 bytes
Total is 132 Bytes

3 Rows
Each Row
12 bytes

? name = 'SAM'

Total
36 Bytes
emp1

2 Rows
Each Row
10 bytes

? dname = 'SBST'

Total
20 Bytes
dep1

Access From Disk
- 56 Bytes

Viewing The query Plan

PLAN_TABLE_OUTPUT

Plan hash value: 2561979186

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		6	132	7 (0)	00:00:01
1	MERGE JOIN CARTESIAN		6	132	7 (0)	00:00:01
* 2	TABLE ACCESS FULL	DEP1	2	20	3 (0)	00:00:01
3	BUFFER SORT		3	36	4 (0)	00:00:01
* 4	TABLE ACCESS FULL	EMP1	3	36	2 (0)	00:00:01

PLAN_TABLE_OUTPUT

Temp1

ID	NAM	DEP	DES	DNO	DNAM	DLO
---	---	---	---	---	----	---
123	SAM	111	AKJ	111	SBST	SJT
126	SAM	222	CMU	111	SBST	SJT
127	SAM	333	KCM	111	SBST	SJT
123	SAM	111	AKJ	444	SBST	MBD
126	SAM	222	CMU	444	SBST	MBD
127	SAM	333	KCM	444	SBST	MBD

? Dep_no = dno (Temp1)

Temp1 is reduced to 132 bytes
from 440 Bytes.



Name,Dname(Temp2)

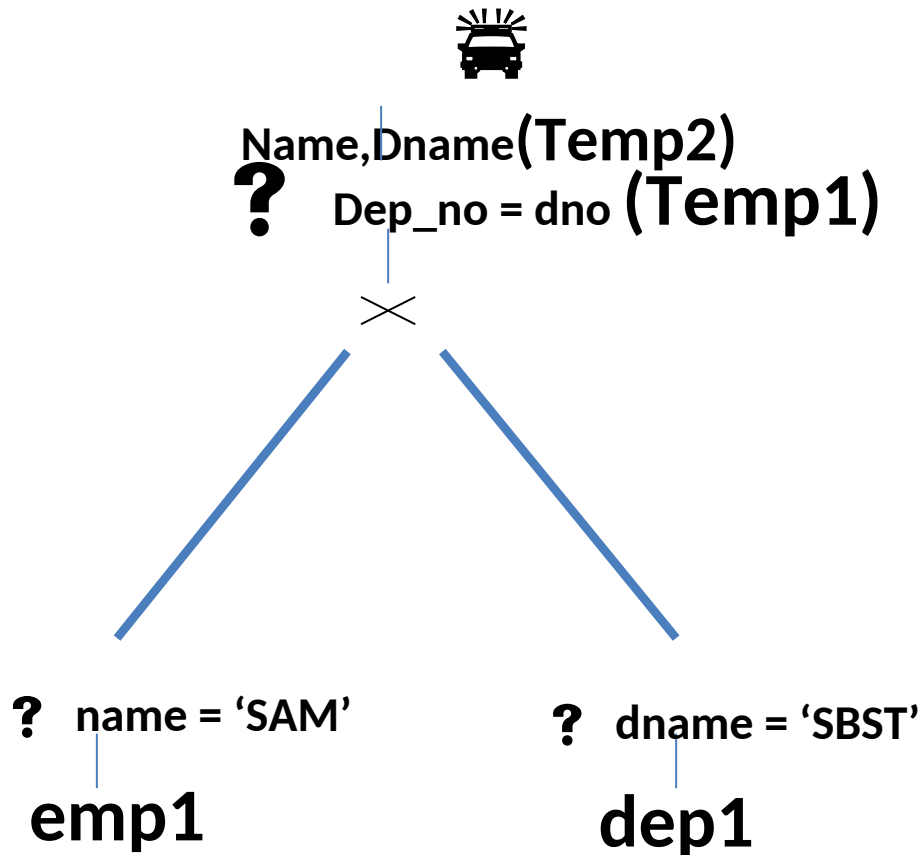
ID	NAM	DEP	DES	DNO	DNAM	DLO
---	---	---	---	---	-----	---
123	SAM	111	AKJ	111	SBST	SJT

Final Result

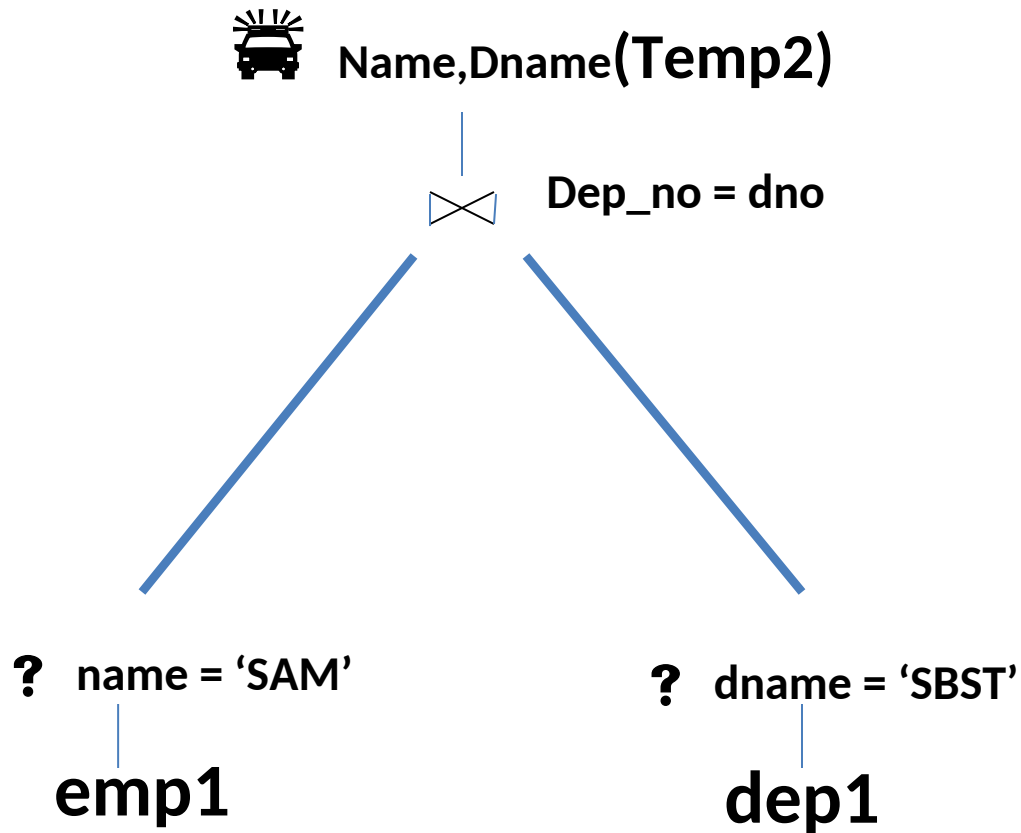
NAM	DNAM
---	-----
SAM	SBST

RULE 3

- Combine the Cartesian Product with a subsequent select operation to a Join Operation.



Rule 3 : Combine the Cartesian Product with a subsequent select operation to a Join Operation



EMP1

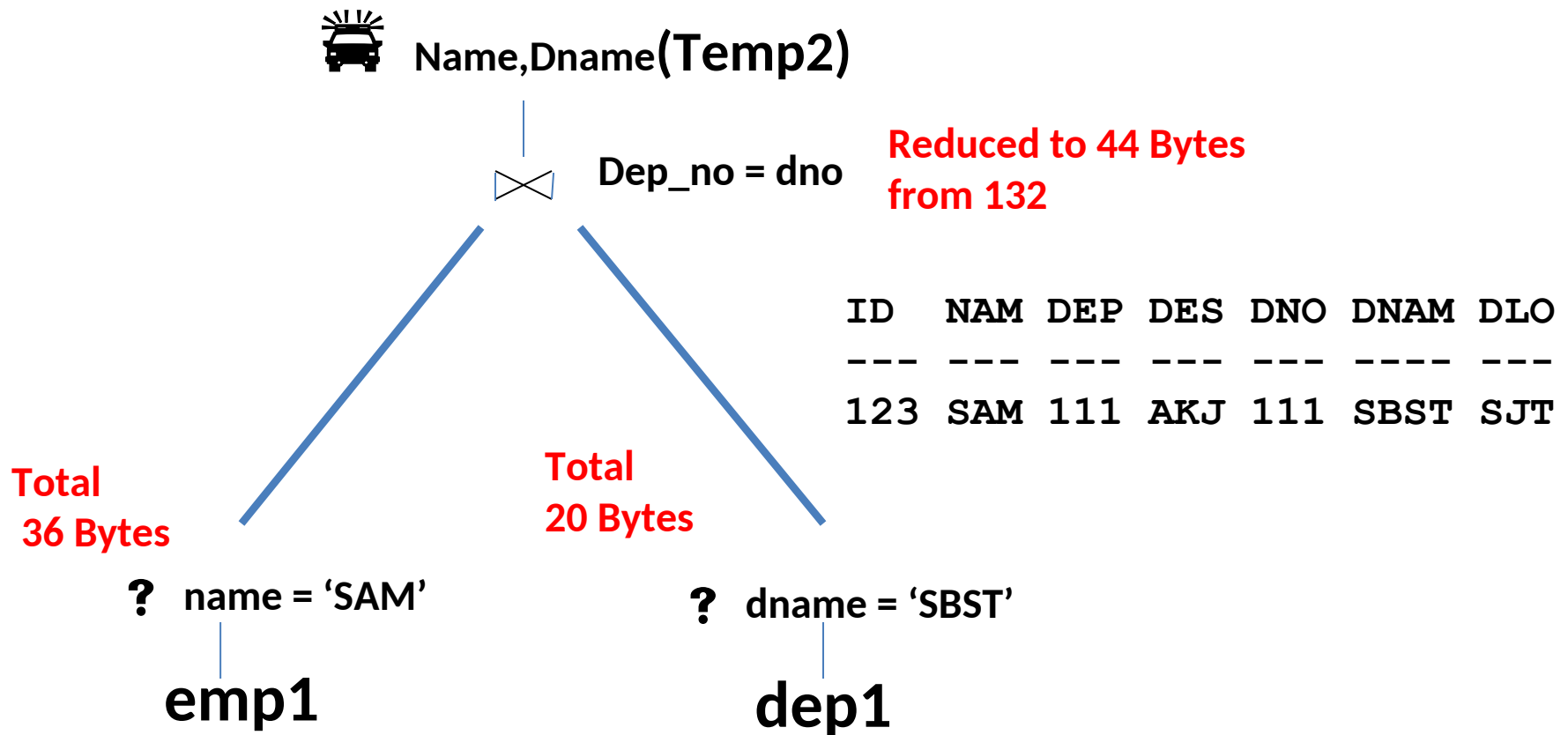
ID	NAM	DEP	DES
---	---	---	---
123	SAM	111	AKJ
126	SAM	222	CMU
127	SAM	333	KCM

DEP1

DNO	DNAM	DLO
---	---	---
111	SBST	SJT
444	SBST	MBD



Rule 3 : Combine the Cartesian Product with a subsequent select operation to a Join Operation



Rule 4

- Break Down the Project operations down the tree as far as possible.

```
SQL> SELECT * FROM EMP1;
```

ID	NAM	DEP	DES
---	---	---	---
123	SAM	111	AKJ
124	TOM	222	CMU
125	RAM	333	KCM
126	SAM	222	CMU
127	SAM	333	KCM

DEP1 Table

DNO	DNAM	DLO
-----	------	-----

---	-----	---
-----	-------	-----

111	SBST	SJT
-----	------	-----

222	SITE	MBD
-----	------	-----

333	SMEC	SMV
-----	------	-----

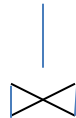
444	SBST	MBD
-----	------	-----

Break the Project Operations
and Move it down the tree

Rule 4 : Break and Move Project Operations down the tree as far as possible



Name,Dname



Dep_no = dno

Total
26 Bytes

NAM DEP

--- ---

SAM 111

SAM 222

SAM 333

DNO DNAM

--- ----

111 SBST

444 SBST

Total
18 Bytes



Name,Depno

Total
14 Bytes



dno,dname

Total
20 Bytes



dname = 'SBST'

Total
36 Bytes



name = 'SAM'

emp1

dep1

Total
60 Bytes

Total
40 Bytes

Execution Plan

Plan hash value: 4277020463

	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
	0	SELECT STATEMENT		2	26	7 (15)	00:00:01
*	1	HASH JOIN		2	26	7 (15)	00:00:01
*	2	TABLE ACCESS FULL	DEP1	2	14	3 (0)	00:00:01
*	3	TABLE ACCESS FULL	EMP1	3	18	3 (0)	00:00:01

Another Exercise

Display the name for all employees who work on a project by name OSP.

`emp1`

ID	NAM	DEP	DES
---	---	---	---
123	SAM	111	AKJ
124	TOM	222	CMU
125	RAM	333	KCM
126	SAM	222	CMU
127	SAM	333	KCM

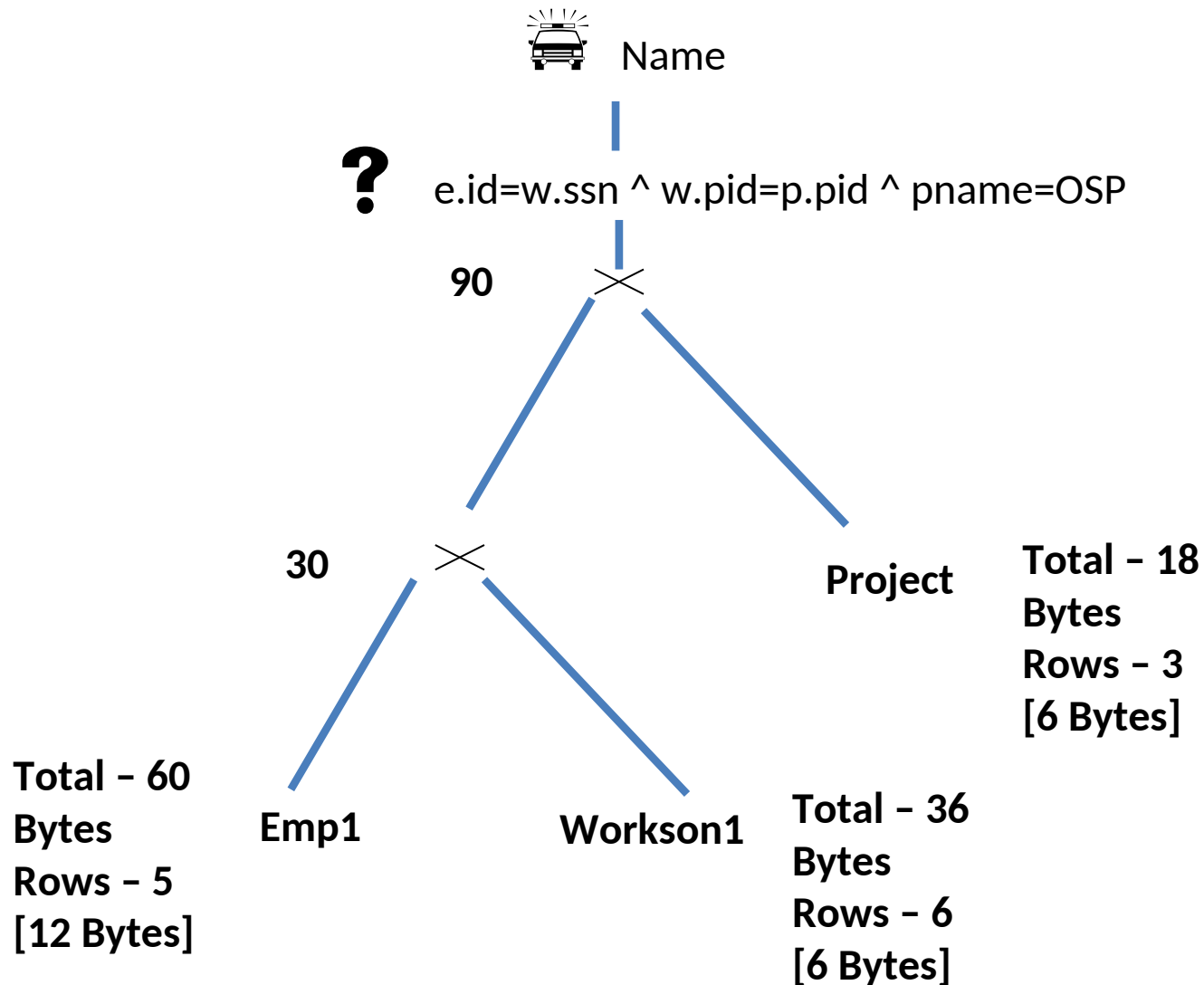
`workson1`

SSN	PID
---	---
111	DBM
123	DBM
124	OSP
125	DBM
126	NET
127	OSP

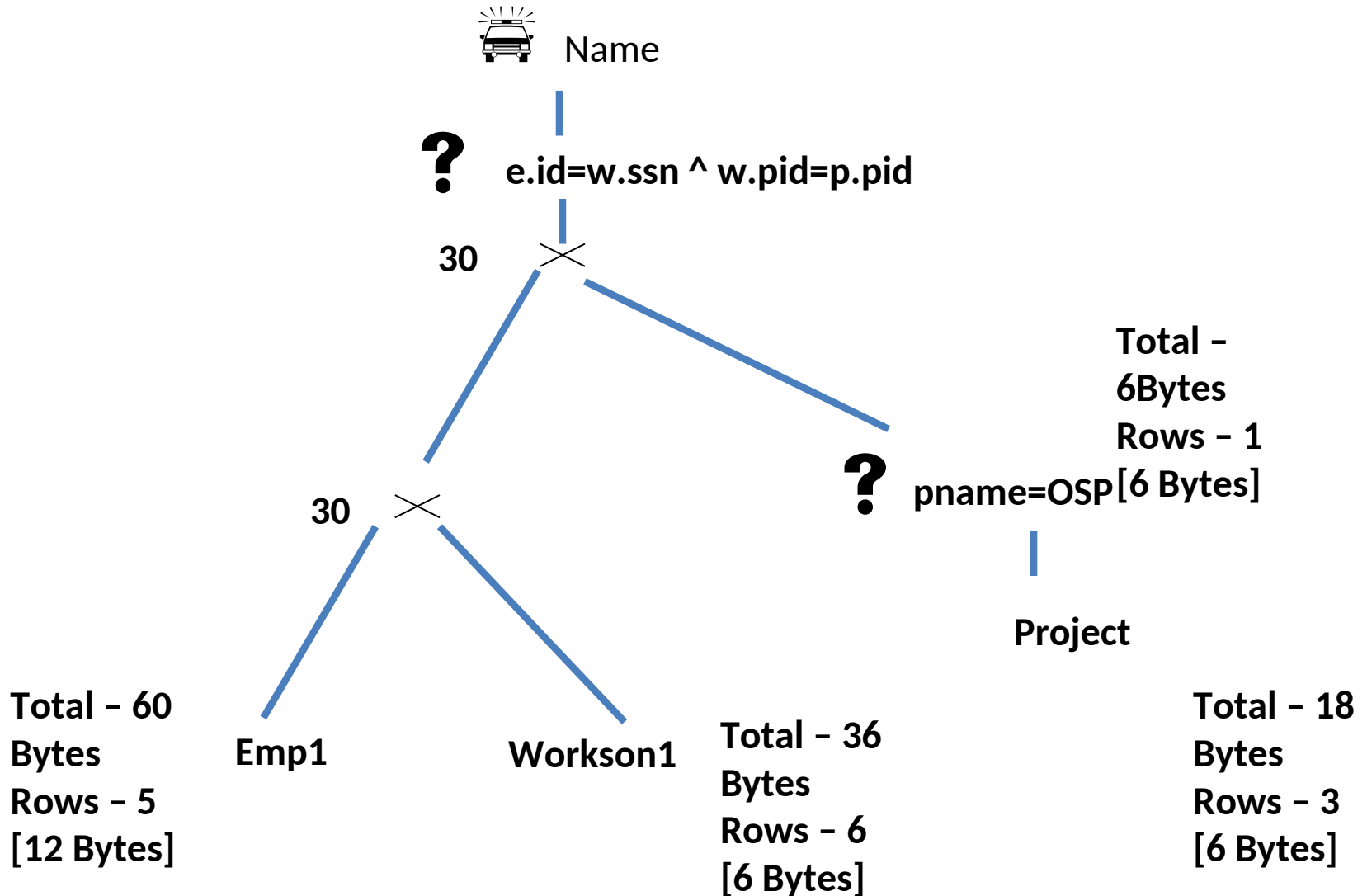
`project1;`

PID	PNA
---	---
DBM	DAT
OSP	OPE
NET	COM

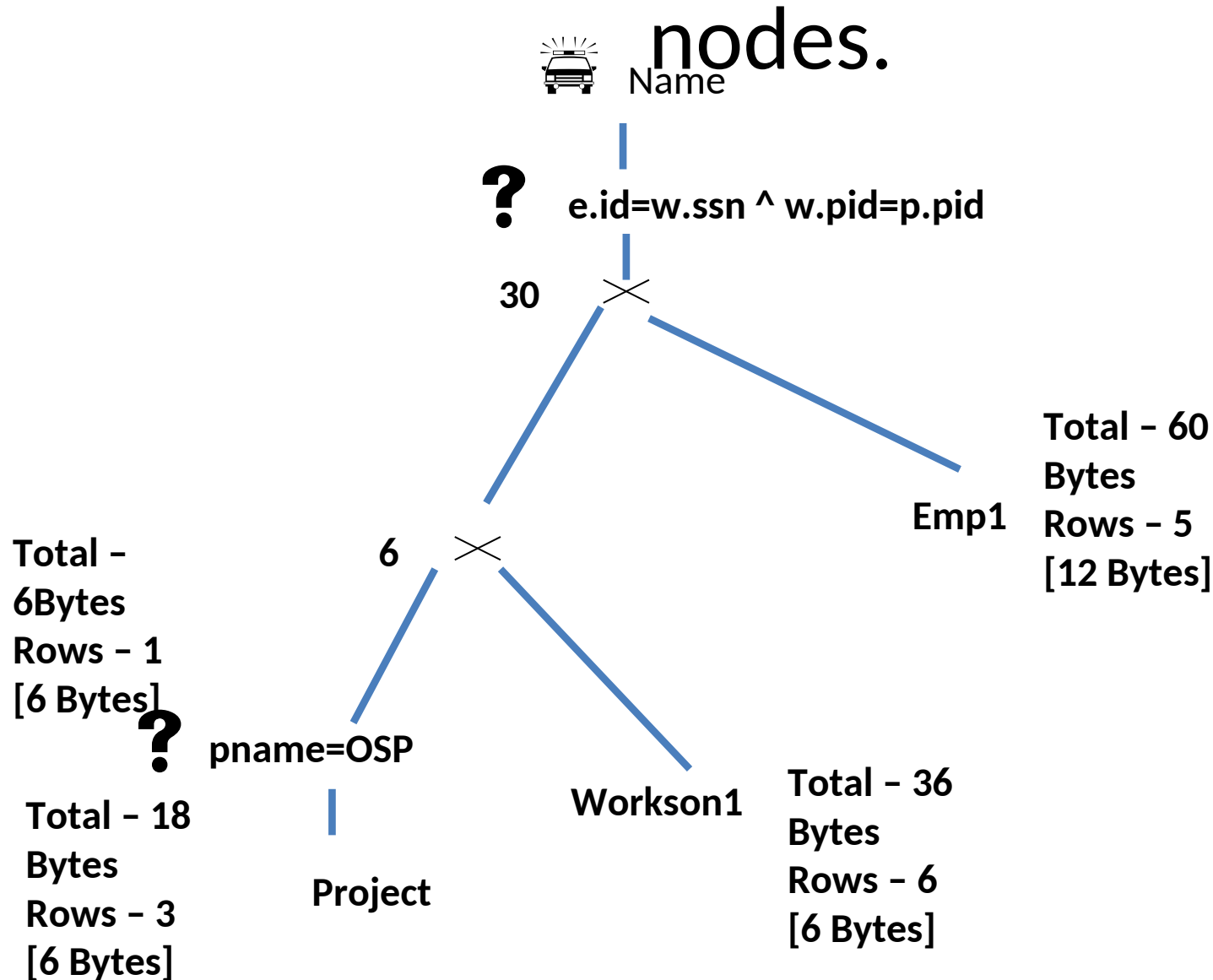
Initial Query Tree



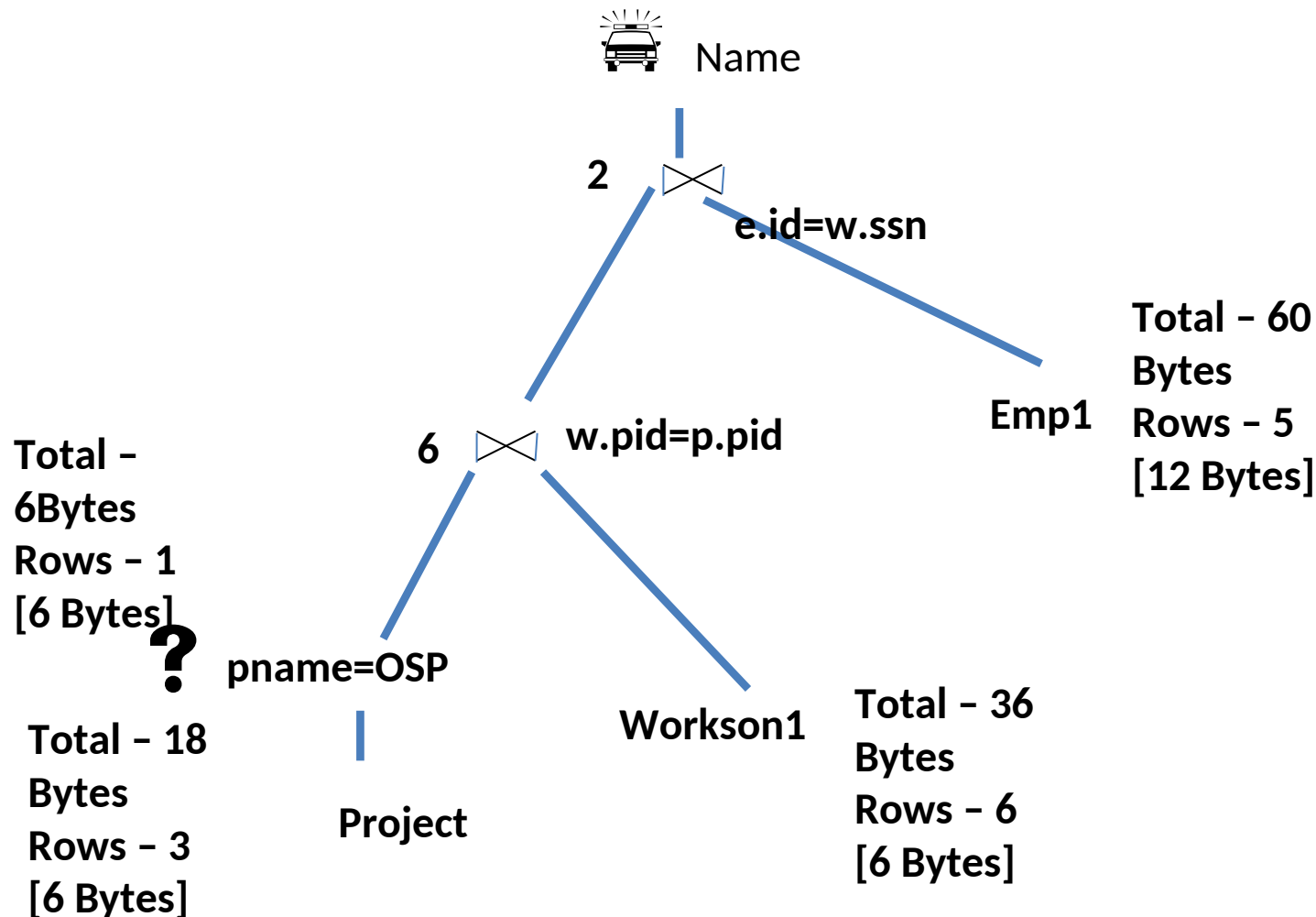
Break and Move Selection Down the Tree (Rule 1 and Rule 2)



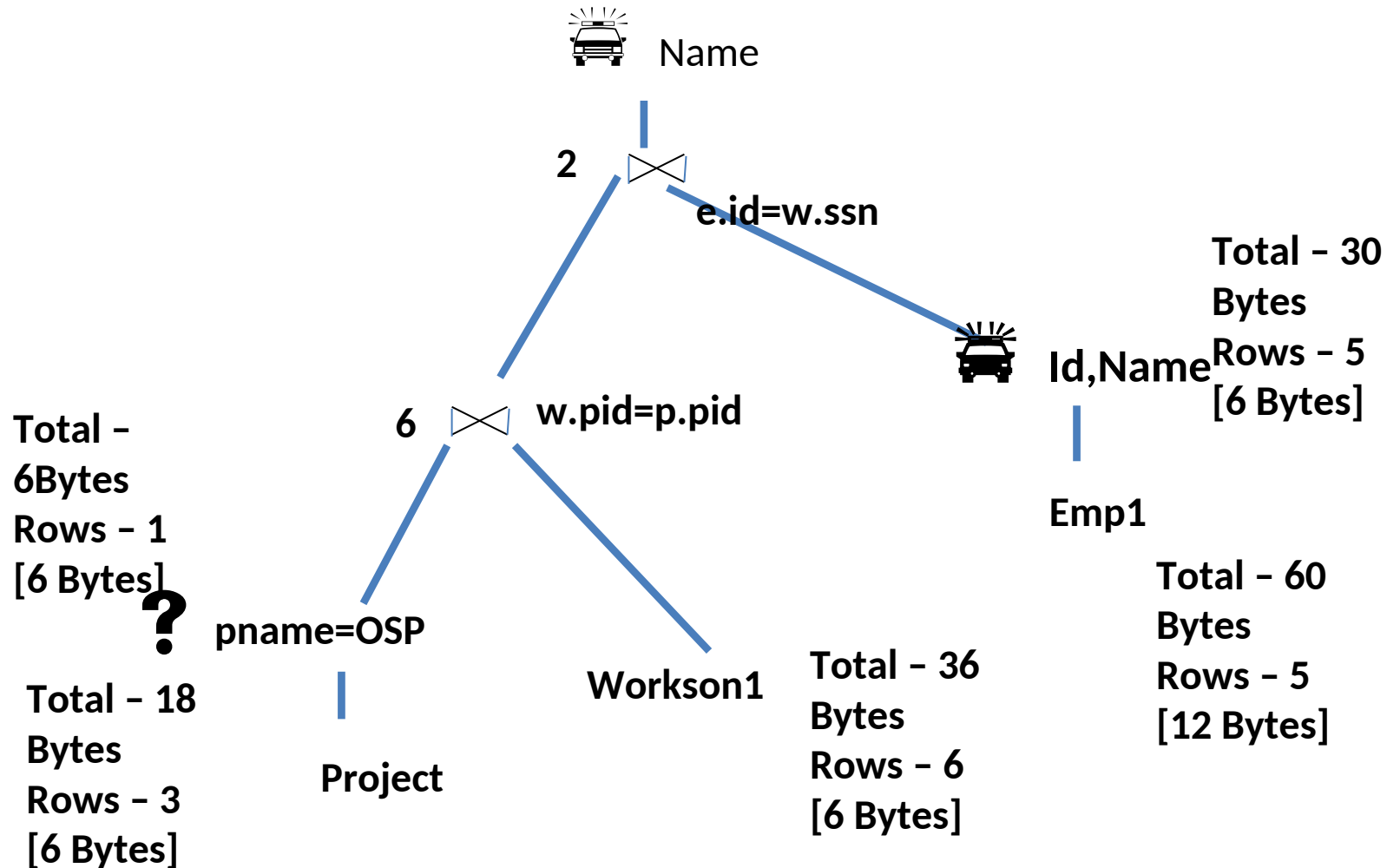
Rule 3- Most Restrictive Select should be done first – Rearrange the leaf



Rule 4- Replace Cartesian Product with Join.



Rule 5- Move Project Operations Down the tree



General Transformation Rules

- General Transformation Rules for Relational Algebra Operations:
 1. Cascade of **S**: A conjunctive selection condition can be broken up into a cascade (sequence) of individual **S** operations:
 - $S_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } c_n}(R) = S_{c1} (S_{c2} (\dots (S_{c_n}(R)) \dots))$
 2. Commutativity of **S**: The **S** operation is commutative:
 - $S_{c1} (S_{c2}(R)) = S_{c2} (S_{c1}(R))$
 3. Cascade of **p**: In a cascade (sequence) of **p** operations, all but the last one can be ignored:
 - $p_{List1} (p_{List2} (\dots (p_{Listn}(R)) \dots)) = p_{List1}(R)$
 4. Commuting **S** with **p**: If the selection condition *c* involves only the attributes *A1*, ..., *An* in the projection list, the two operations can be commuted:
 - $p_{A1, A2, \dots, An} (S_c (R)) = S_c (p_{A1, A2, \dots, An} (R))$

General Transformation Rules

- General Transformation Rules for Relational Algebra Operations (contd.):
- Commutativity of $\bowtie$ ($\bowtie_c$ and $\times$): The $\bowtie$ operation is commutative as is the $\times$ operation:
 - $R \bowtie_c S = S \bowtie_c R$; $R \times S = S \times R$
 - Commuting S with $\bowtie$ (or $\times$): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R —the two operations can be commuted as follows:
 - $S_c (R \bowtie S) = (S_c (R)) \bowtie S$
- Alternatively, if the selection condition c can be written as (c_1 and c_2), where condition c_1 involves only the attributes of R and condition c_2 involves only the attributes of S , the operations commute as follows:
 - $S_c (R \bowtie S) = (S_{c_1} (R)) \bowtie (S_{c_2} (S))$

General Transformation Rules

- General Transformation Rules for Relational Algebra Operations (contd.):
- 7. Commuting ρ with $\bowtie$ (or $\times$): Suppose that the projection list is $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, where $A_1, \dots, A_n$ are attributes of R and $B_1, \dots, B_m$ are attributes of S . If the join condition c involves only attributes in L , the two operations can be commuted as follows:
 - $\rho_L (R \bowtie_C S) = (\rho_{A_1, \dots, A_n} (R)) \bowtie_C (\rho_{B_1, \dots, B_m} (S))$
- If the join condition C contains additional attributes not in L , these must be added to the projection list, and a final ρ operation is needed.

General Transformation Rules

- General Transformation Rules for Relational Algebra Operations (contd.):
8. Commutativity of set operations: The set operations $\cup$ and $\cap$ are commutative but “ $-$ ” is not.
 9. Associativity of $\bowtie$, $\times$, $\cup$, and $\cap$: These four operations are individually associative; that is, if q stands for any one of these four operations (throughout the expression), we have
 - $(R \ q \ S) \ q \ T = R \ q \ (S \ q \ T)$
 10. Commuting s with set operations: The s operation commutes with $\cup$, $\cap$, and $-$. If q stands for any one of these three operations, we have
 - $s_c (R \ q \ S) = (s_c (R)) \ q \ (s_c (S))$

Using Heuristics in Query Optimization

1. Using rule 1, break up any select operations with conjunctive conditions into a cascade of select operations.
2. Using rules 2, 4, 6, and 10 concerning the commutativity of select with other operations, move each select operation as far down the query tree as is permitted by the attributes involved in the select condition.
3. Using rule 9 concerning associativity of binary operations, rearrange the leaf nodes of the tree so that the leaf node relations with the most restrictive select operations are executed first in the query tree representation.
4. Using Rule 12, combine a Cartesian product operation with a subsequent select operation in the tree into a join operation.
5. Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.
6. Identify subtrees that represent groups of operations that can be executed by a single algorithm.

Another Exercise

- Select name ,project name for all employees who work in department by name 'SITE' and Project name ='OSP'
- In addition to EMP1 and DEP1 we have Project table and workson table that maps Employees to Projects.

```
SQL> SELECT * FROM WORKSON1;
```

SSN	PID
---	---
111	DBM
123	DBM
124	OSP
125	DBM
126	NET
127	OSP

```
SELECT * FROM PROJECT1;
```

```
SQL> SELECT * FROM PROJECT1;
```

```
PID  PNA
```

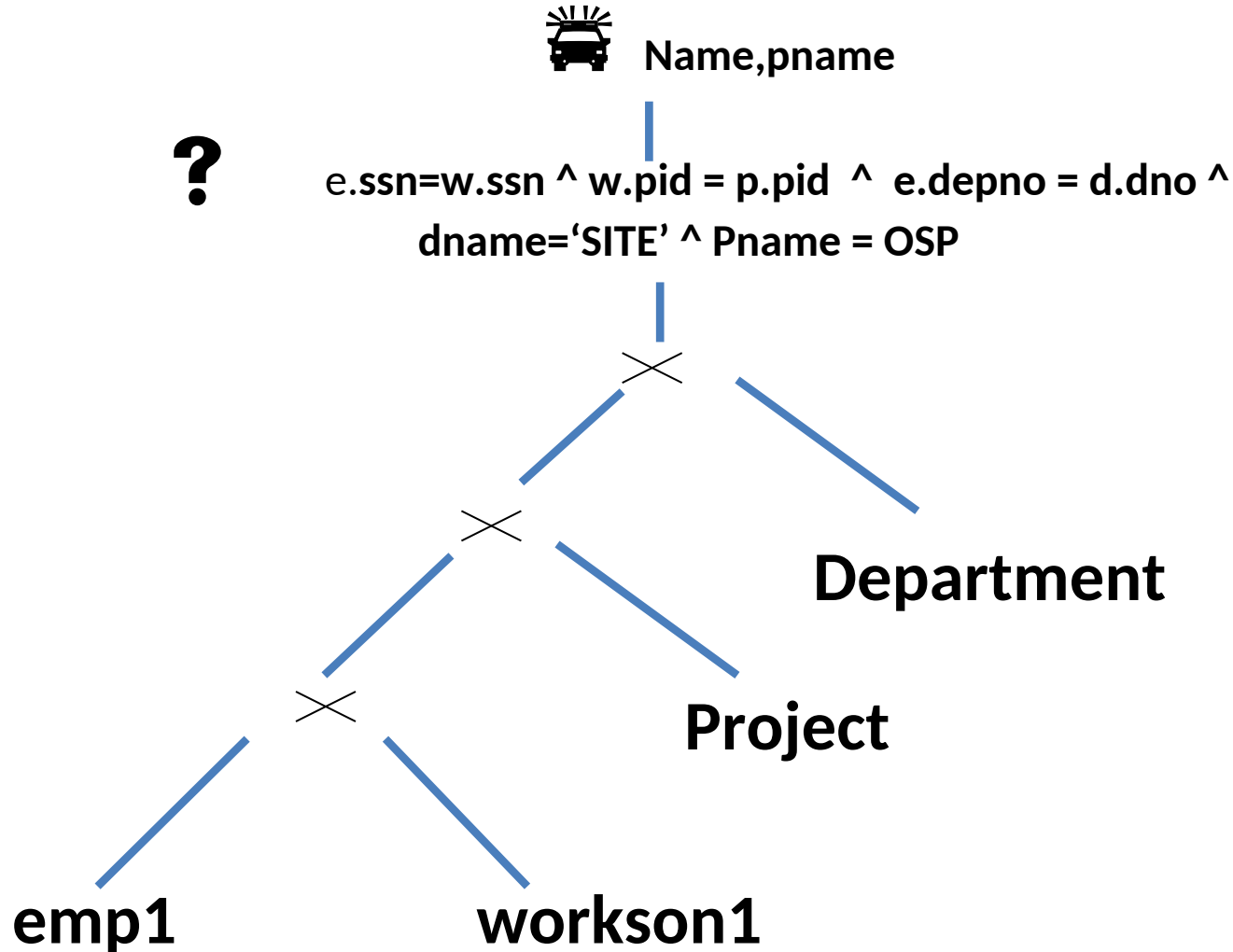
```
---  ---
```

```
DBM  DAT
```

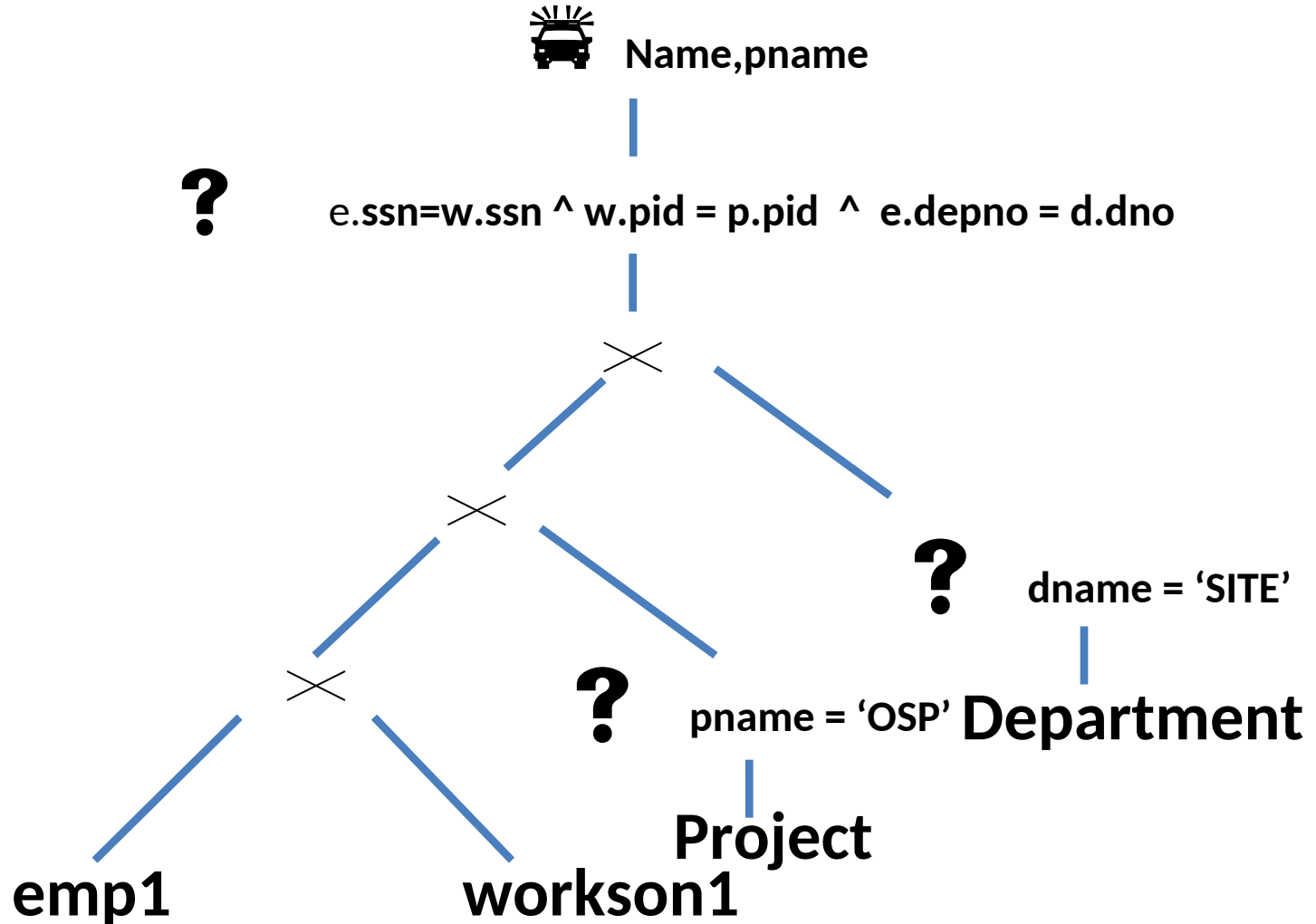
```
OSP  OPE
```

```
NET  COM
```

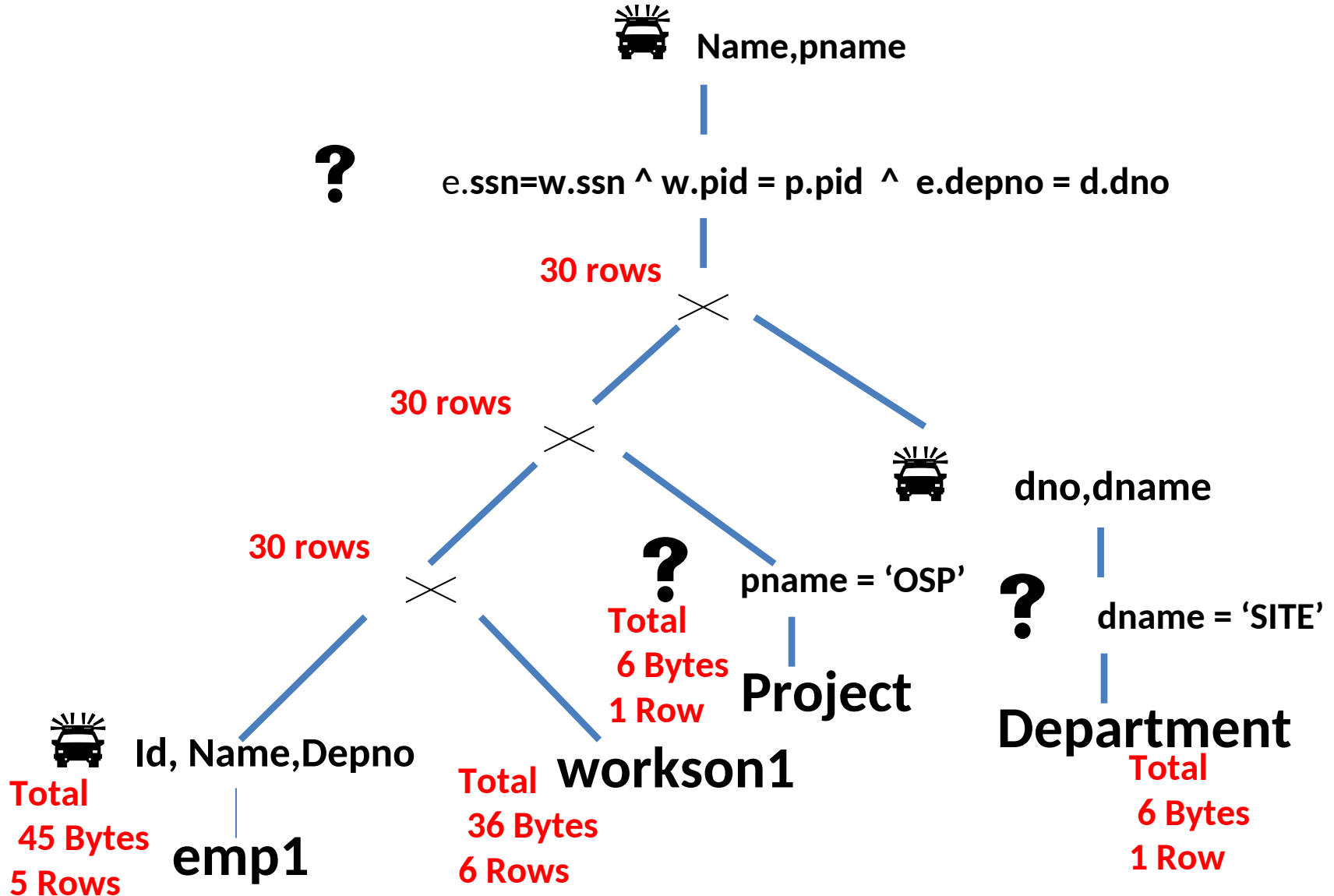
Initial Query Tree



Moving Selection down the tree



Moving projection down the tree



Rearrange
leaf nodes
of the tree

